

SIMATS ENGINEERING ASSESSMENT TOOL

COURSE NAME: Theory of Computation COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ϵ -NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:20 Duration : 1 Hour

Scenario Based

Code of Conduct:

I M.Vinay(192311146)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action

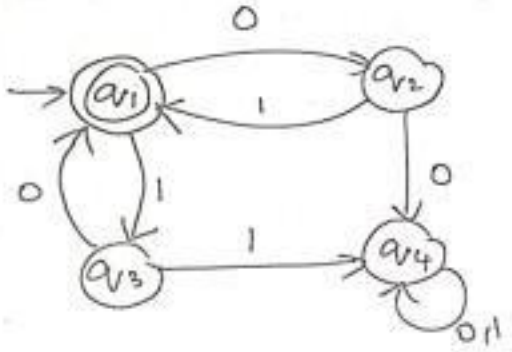
M.Vinay

Signature of the student:

SIMATS ENGINEERING ASSESSMENT TOOL

1. Given a DFA used for validating a specific PIN pattern bbbc, bca ,c, ca, caaaaa minimize the DFA and explain how state reduction improves performance and efficiency in embedded systems

2. Demonstrate how a given finite automaton can be converted into a regular expression



3. Consider the following

grammar $S \rightarrow aSc | T | U | e$

$T \rightarrow bTc | e$

$U \rightarrow aUb |$

e

A) What is the language defined by the grammar?

B) Give the derivation tree for the word aaabbc

C Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

4. I) Show that if L_1 and L_2 are regular then $(L_1 + L_2)$ also

regular II) Is the language $\{ a^p \mid p \text{ is prime} \}$ regular? Justify

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Answer

1. Accepted Language

$L = \{bbbc,; bca,; c,; ca,; caaaaaaa\}$

where the alphabet is

$\Sigma = \{a,b,c\}$

Minimized DFA

A minimized DFA merges equivalent states that have identical future behavior.

Since the accepted strings share common prefixes:

- **c**, **ca**, and **caaaaaaa** share the prefix **c**.
- **bbbc** shares the prefix **bbb**.
- **bca** shares the prefix **bc**.

Equivalent states are merged, while keeping a single **dead state** for all invalid inputs.

The minimized DFA therefore contains:

- Start state
- States for the common prefixes (**b**, **bb**, **bbb**, **bc**, **c**, **ca**, ...)
- Accepting states for the required PINs
- One dead (trap) state

Transition Table (Conceptual)

State	Purpose
q0	Start state
q1	Reads b
q2	Reads bb
q3	Reads bbb
q4	Accepts bbbc

q5	Reads bc
q6	Accepts bca
q7	Accepts c
q8	Accepts ca
q9–q14	Read additional as for caaaaaaa
q15	Accepts caaaaaaa
qd	Dead state

After minimization, equivalent states (if any) are merged and only **one dead state** is retained.

How State Reduction Improves Performance

Minimizing the DFA provides several benefits:

- **Reduces memory usage** by storing fewer states.
- **Reduces hardware cost** by requiring fewer logic gates and flip-flops.
- **Increases execution speed** because fewer state transitions are processed.
- **Consumes less power**, making it suitable for embedded and battery-powered devices.
- **Simplifies implementation and testing**, improving system reliability.

Applications

Minimized DFAs are commonly used in:

- PIN validation in ATM machines
- Embedded security systems
- Digital door locks
- Smart cards
- Authentication modules
- Access control systems

2.Demonstrate how a given finite automaton can be converted into a regular expression.

Write the transitions

From the automaton:

- $(q_1 \rightarrow \{0\} q_2)$
- $(q_2 \rightarrow \{1\} q_1)$
- $(q_1 \rightarrow \{1\} q_3)$
- $(q_3 \rightarrow \{0\} q_1)$
- $(q_2 \rightarrow \{0\} q_4)$
- $(q_3 \rightarrow \{1\} q_4)$
- $(q_4 \rightarrow \{0,1\})$

q_4) where:

- q_1 is the **start state**
- q_1 is also the **final state**
- q_4 is the **trap state**

Observe the accepted

patterns From (q_1) :

- Reading **01** returns to (q_1) .
- Reading **10** also returns to (q_1) .
- Reading **00** or **11** leads to the trap state (q_4) .

Hence, only **01** and **10** can be repeated.

Write the Regular Expression

$(01|10)^*$

The equivalent regular expression is

$(01+10)^*$

because the automaton repeatedly accepts the patterns **01** and **10**, while **00** and **11** move the automaton to the trap state.

Regular Expression:

$\{(01+10)^*\}$

1. The start state (q_1) is also the accepting state.
2. The transitions **01** and **10** return the automaton to the accepting state.
3. The transitions **00** and **11** lead to the trap state (q_4), where all further inputs are rejected.
4. Therefore, the equivalent regular expression is: $01+10)^*$

Question 3

Given Grammar:

$STU \rightarrow aSc \mid T \mid U \mid \varepsilon \rightarrow bTc \mid \varepsilon \rightarrow aUb \mid \varepsilon$

A) What is the language defined by the grammar?

Analyze each production separately.

1. Production ($T \rightarrow bTc \mid \varepsilon$)

Generates:

- ε
- bc
- $bbcc$
- $bbbccc$
- ...

Therefore,

$L(T) = \{b^n c^n \mid n \geq 0\}$

2. Production ($U \rightarrow aUb \mid \varepsilon$)

Generates:

- ϵ
- ab
- aabb
- aaabbb
- ...

Therefore, $L(U) = \{anbn \mid n \geq 0\}$

3. Production ($S \rightarrow aSc$)

Each application adds

- one **a** at the beginning
- one **c** at the end.

Finally, (S) becomes either (T), (U), or ϵ .

Hence the language is

$$L = \{a^i b^j c^{i+j} \mid i, j \geq 0\} \cup \{a^{i+j} b^j c^i \mid i, j \geq 0\}$$

B) Derivation Tree for the string

aaabbc Leftmost Derivation

S

$\Rightarrow aSc$

$\Rightarrow aUc$

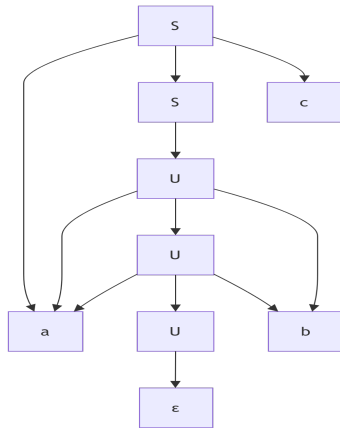
$\Rightarrow aaUbc$

$\Rightarrow aaaUbbc$

$\Rightarrow aaa\epsilon bbc$

$\Rightarrow aaabbc$

Derivation Tree



Reading the leaves from left to right gives:

a a a b b c

which is

aaabbc

C) Is the grammar ambiguous? Justify.

Definition

A grammar is **ambiguous** if a string has two or more different derivations (or parse trees).

Consider the string ϵ

It can be derived in three different ways:

$$S \Rightarrow \varepsilon$$

$$S \Rightarrow T \Rightarrow$$

$$\varepsilon S \Rightarrow U$$

$$\Rightarrow \varepsilon$$

Since the same string ε has more than one valid derivation, The grammar is ambiguous.